CORPORACIÓN UNIFICADA NACIONAL CUN

RETOS ALGORÍTMICOS

Título: Retos Algorítmicos - Fundamentos de Programación

Nombre de los autores:

Diana Paola Parra Espinosa 55557

Docente: Amalia Castro Ardila

8 de noviembre de 2026

Contenido

Introducción 4

Objetivo General 5

Objetivos Específicos 5

Elección de la propuesta 5

Algoritmos 5

Reto 05 - Simulador de Recargo Nocturno y Horas Extras (proyecto final) 6

Pseudocódigo 9

Descripción General de la implementación 10

Link de Video 10

Repositorio GitHub 11

Conclusiones 11

Referencias 11

Introducción

El presente documento describe el trabajo realizado en la asignatura Fundamentos de Programación, correspondiente a la resolución de 40 retos algorítmicos organizados en 8 módulos, más un proyecto final (Reto 05 - Simulador de Recargo Nocturno y Horas Extras). Los retos se desarrollaron en Python 3 (Python Software Foundation, 2026) y abordan situaciones cotidianas del estudiante universitario (transporte, finanzas personales, estudio, salud, alimentación, entre otros), aplicando estructuras de control, funciones, listas, diccionarios, tuplas y ciclos. Cada reto incluye una solución de referencia ejecutable con casos de prueba.

Objetivo General

Desarrollar el pensamiento algorítmico y las habilidades de programación en Python mediante la resolución de 40 retos algorítmicos y un proyecto final, aplicando buenas prácticas de diseño de funciones, validación de datos y pruebas.

Objetivos Específicos

- Analizar problemas cotidianos y traducirlos a algoritmos implementados en Python.
- Implementar funciones reutilizables con parámetros y valores de retorno.
- Usar listas, diccionarios, tuplas y conjuntos para modelar datos.
- Aplicar ciclos, condicionales y operadores para resolver cálculos.
- Validar entradas del usuario y manejar casos límite.
- Documentar y probar las soluciones con casos de prueba.

Elección de la propuesta

Como proyecto final se eligió el Reto 05 - Simulador de Recargo Nocturno y Horas Extras (Módulo 1, nivel intermedio). La elección se justifica porque integra en una sola solución los conceptos centrales del curso: funciones, listas, diccionarios y ciclos, además de validación de entradas y pruebas. El reto modela un caso real de liquidación de salario quincenal para un trabajador de comercio/call center, aplicando recargos nocturno (35%) y festivo (75%) y un descuento de ley del 8%.

Algoritmos

Reto 05 - Simulador de Recargo Nocturno y Horas Extras (proyecto final)

Entradas: salario por hora, horas diurnas, horas nocturnas (después de las 7 p.m.) y horas festivas. Cálculo del salario bruto: bruto = salario_hora * diurnas + salario_hora * 1.35 * nocturnas + salario_hora * 1.75 * festivas. Salario neto: neto = bruto * (1 - 0.08), redondeado a 2 decimales. Validaciones: el total de horas no puede superar 240 (quincena) y debe ser mayor que cero.

Tabla 1

Resumen de los 40 retos algorítmicos

Reto	Título	Conceptos principales
01	Optimizador de Ruta TransMilenio en Hora Pico	Condicionales, multiplicación
02	Calculadora de Costos en Taxis vs. Plataformas	Diccionarios, min, filtros
03	Validador de Subsidio y Tarjeta TuLlave (SITP)	Diccionarios, max
04	Monitoreo de Bici-parqueaderos y Batería	Condicionales
05	Simulador de Recargo Nocturno y Horas Extras	Funciones, aritmética
06	Presupuesto Quincenal del Estudiante-Trabajador	Listas, sum, slicing
07	Vaca Virtual para Proyectos de Aula	Sum, listas, condicionales
08	Comparador de Combo Comida Rápida Nocturna	min con clave compuesta
09	Calculadora de Nota Mínima Requerida (Tercer Corte)	Fórmula, max

Reto	Título	Conceptos principales
10	Detector de Cruce de Horarios y Disponibilidad	Sets, unión, filtrado
11	Alerta de Inasistencia (Regla del 20%)	Porcentaje, max, int
12	Optimizador de Micro-tiempos de Estudio en el Bus	Fuerza bruta (máscaras de bits)
13	Ruta Segura y Mapeo de Zonas Iluminadas	max con clave ponderada
14	Sistema de Red de Apoyo / Paradero Seguro	Diccionarios, agrupación
15	Simulador de Reporte de Extravío de Documentos	Condicionales, acumulador
16	Disponibilidad de Salas de Cómputo e Impresiones	Aritmética, inf
17	Gestión de Parqueaderos de Motos y Carros	Condicionales, tarifas
18	Planificador de Alimentación y Consumo de Cafeína	Alertas, condicionales
19	Cálculo de Huella de Carbono del Desplazamiento	Diccionarios, sum
20	Monitor de Clima y Paso por Encharcamientos	Listas, condicionales
21	Optimización de Rutas de Entrega Nocturna	Ordenamiento greedy
22	Gestión de Inventario en Minimarkets Cercanos	Listas, max
23	Calculadora de Costos Compartidos en Uber Pool	Proporciones, diccionarios

Reto	Título	Conceptos principales
24	Planificador de Turnos para Call Center Nocturno	Aritmética, medianoche
25	Control de Préstamos y Penalizaciones de Biblioteca	max, aritmética
26	Simulador de Matrícula Sabatina vs. Nocturna	sum, condicionales
27	Validador de Prerrequisitos del Plan de Estudios	Conjuntos, subconjunto
28	Evaluador del Impacto de Cancelar Materia	Promedio ponderado
29	Monitor de Deuda de Sueño Semanal	Racha, sum
30	Control de Hidratación en la Jornada Continua	Meta, porcentaje
31	Evaluador del Nivel de Estrés por Carga Combinada	sum, condicionales
32	Contador de Pasos y Distancia en Transbordos	sum, conversión
33	Simulador de Batería y Ahorro para Emergencias	Condicionales, listas
34	Registro y Clasificación de Incidencias en Campus	Filtrado por hora
35	Validador de Identidad y Control de Acceso	Conjuntos, contador
36	Calculadora de Costos por Pérdida de Objetos	sum, ordenamiento
37	Simulador de Arriendo Compartido entre Estudiantes	Proporciones por área

Reto	Título	Conceptos principales
38	Cálculo de Consumo de Datos en Clases Híbridas	División, min
39	Planificador de Compra en Merca Nocturna	Greedy por precio
40	Eficiencia de Vehículo / Moto en Tráfico Bogotano	Rendimiento, costos

Nota. Elaboración propia a partir de los archivos reto_01.py a reto_40.py del curso.

Pseudocódigo

Pseudocódigo del proyecto final (Reto 05):

```
FUNCIÓN calcular_salario_bruto(salario_hora, diurnas, nocturnas, festivas):
   RETORNAR salario_hora*diurnas + salario_hora*1.35*nocturnas +
salario_hora*1.75*festivas

FUNCIÓN calcular_salario_neto(salario_hora, diurnas, nocturnas, festivas, descuento=0.08):
   bruto = calcular_salario_bruto(...)
   RETORNAR redondear(bruto*(1-descuento), 2)

FUNCIÓN validar_horas(diurnas, nocturnas, festivas, tope=240):
   total = diurnas + nocturnas + festivas
   SI total > tope: RETORNAR (FALSO, "supera el tope")
   SI total == 0: RETORNAR (FALSO, "debe ingresar horas")
   RETORNAR (VERDADERO, "OK")

MENÚ:
   MIENTRAS verdadero:
      MOSTRAR opciones 1) Calcular 2) Pruebas 3) Salir
      LEER opción
      SI opción == "1": leer datos, validar, mostrar resumen
      SI opción == "2": ejecutar 5 casos de prueba
      SI opción == "3": SALIR
```

Los 40 retos restantes siguen el mismo patrón: una función pura que recibe parámetros y retorna un resultado, con casos de prueba en el bloque principal. El pseudocódigo detallado de cada reto se encuentra en su archivo fuente (reto_XX.py).

Descripción General de la implementación

Los retos se implementaron en Python 3, organizados en 8 módulos de 5 retos cada uno (archivos reto_XX.py) más el proyecto final en proyecto_final/src/main.py. Cada reto es una función pura con casos de prueba en el bloque principal. El proyecto final se estructura en funciones de cálculo, validación de datos y un menú interactivo por consola. La estructura sigue las orientaciones del material del curso (Corporación Unificada Nacional CUN, s.f.).

Tabla 2

Casos de prueba del proyecto final (Reto 05)

Caso	Sal/Hora	Diurnas	Nocturnas	Festivas	Neto esperado
1	8000	60	20	4	691840.0
2	10000	80	0	0	736000.0
3	9000	0	40	0	447120.0
4	7500	0	0	10	120750.0
5	8500	30	10	5	408595.0

Nota. Valores base: recargo nocturno +35%, recargo festivo +75%, descuento de ley 8%. Fórmula: bruto = (H_diurnas + 1.35*H_nocturnas + 1.75*H_festivas) * salario_hora; neto = bruto * 0.92.

Link de Video

Enlace de sustentación: https://drive.google.com/file/d/1n-SLNhFxUUZi35U0-qDwHsQ2mtFXTw5Y/view?usp=drive_link

El código fuente completo del proyecto (40 retos + proyecto final) está disponible en: https://github.com/dianaparraee/retos_algoritmicos

Repositorio GitHub

Conclusiones

- Se fortaleció el pensamiento algorítmico al modelar problemas cotidianos en Python.
- Se aplicaron funciones, listas, diccionarios y ciclos de forma integrada en el proyecto final.
- La validación de entradas y las pruebas con casos límite mejoraron la robustez de las soluciones.
- Como mejoras futuras: persistencia de datos, interfaz gráfica, mayor cobertura de pruebas y manejo de errores.

Referencias

Corporación Unificada Nacional CUN. (s.f.). *Material del curso Fundamentos de Programación.*

Python Software Foundation. (2026). *Python 3 documentation.* https://docs.python.org/3/